Waterfall simulation

Natálie Kaslová, Michal Sládek

Department of computer graphics and interaction,
Electrotechnical faculty, CTU Prague

---

1. Introduction

Realistic simulation of fluids is still one of the most resource-intensive processes in computer graphics. It is possible to simulate the surfaces of bodies of water using Gerstner waves, and while the results look great and work in real time, this is far from the generic solution to fluid simulation. In general, fluids can be much more dynamic, which is exactly the case with waterfalls.

Simulating these dynamic fluids requires splitting the continuous fluid into smaller parts that interact with each other. This can be done either by grid-based method that divides space into cells that store velocity and pressure field data, or by particle-based method, such as SPH, that divides the fluid mass into discrete elements (particles). Of course, we need more particles if we want more detail, which in turn requires more computational power and memory. This relationship is quadratic for how many interaction of particles need to be checked and cubic for how many cells we would need to split the 3D space into.

We chose to simulate our waterfall using a method proposed in *Particle-Based Fluid Simulation for Interactive Applications* (2003) by *Matthias Müller, David Charypar and Markus Gross*, which is a modification of the Smoothed particle hydrodynamic (SPH) system.

2. Analysis

2.1. Navier-Stokes equation

The Navier-Stokes equation gives us a description of the motion of incompressible fluids.

$$\rho\left(\frac{\partial \mathbf{v}}{\partial t} + (\mathbf{v}\cdot\nabla)\mathbf{v}\right) = -\nabla p + \rho\mathbf{g} + \mu\nabla^2\mathbf{v}$$

The left hand side of the equation can be simplified to a substantial derivative $D\mathbf{v}/Dt$. Because we are working with particles, and because they move with the fluid, the substantial derivative of the velocity field is simply the time derivative of the velocity of particles. Thus, the convective term $\mathbf{v}.\nabla\mathbf{v}$ is not needed for particle systems.

The right hand side of the equation contains 3 terms used to model force density fields:

- ∇p - pressure
- $\rho\mathbf{g}$ - external forces

- $\mu\nabla^2\mathbf{v}$ - viscosity

The sum of these force density fields determines the change of momentum over time for the particles on the left hand side $\rho\frac{D\mathbf{v}}{Dt}$. So in order for us to calculate the acceleration of particle $\mathbf{a}_i$ we need to divide the force density field by the density field (both at the position of i).

$$\mathbf{a}_i = \frac{\mathbf{f}_i}{\rho_i}$$

2.2. General approach (SPH)

The value of a scalar quantity A_S at position $\mathbf{r}$ is calculated as a weighted sum of contributions of all particles j.

$$A_S(r) = \sum_j m_j \frac{A_j}{\rho_j} W(\mathbf{r} - \mathbf{r}_j, h)$$

- m_j - mass of particle j
- ρ_j - density at the position of particle j
- $\mathbf{r} - \mathbf{r}_j$ - distance between position $\mathbf{r}$ and the position of particle j ($\mathbf{r}_j$)
- W - smoothing kernel function
- h - radius of the smoothing kernel function

Calculating the value of A_S at some arbitrary point in space $\mathbf{r}$ is not very useful to us. Instead, we will calculate the value at the position of a particle i ($\mathbf{r}_i$) and the sum will go over all of the other particles j where $j \neq i$.

2.3. Derivatives

To evaluate $-\nabla p + \rho\mathbf{g} + \mu\nabla^2\mathbf{v}$ we need derivatives of the field quantities. Fortunately, creating a derivative of our general approach equation (2.2) is as simple as using a derivative of the smoothing kernel function.

To get a pressure force, we need the change of the pressure scalar field across space -> 1st derivative (**gradient**).

$$\nabla A_S(r) = \sum_j m_j \frac{A_j}{\rho_j} \nabla W(\mathbf{r} - \mathbf{r}_j, h)$$

To get a viscosity force we need to calculate a "sheer stress" scalar field - how different are the velocities between adjacent elements (1st derivative). Then, we need the change of the sheer stress across space -> 2nd derivative (**Laplacian**).

$$\nabla^2 A_S(r) = \sum_j m_j \frac{A_j}{\rho_j} \nabla^2 W(\mathbf{r} - \mathbf{r}_j, h)$$

2.4. Smoothing kernel function

A function that returns weight based on the distance of 2 points. It is typically implemented using some value that defines the radius in which the particles affect each other. If the distance between 2 particles is greater than the radius, their influence on each other is 0.

Properties of smoothing kernel functions:

- non-negativity - $f(x) \geq 0$ for all x
- symmetry - $f(x) = f(-x)$ for all x
- Normalization - integral is 1

The following smoothing kernel functions are used in the paper:

$$w_{\text{poly6}}(\mathbf{r}, h) = \frac{315}{64\pi h^9} \begin{cases} (h^2 - r^2)^3, & 0 \leq r \leq h, \\ 0, & \text{otherwise} \end{cases}$$

$$w_{\text{spiky}}(\mathbf{r}, h) = \frac{15}{\pi h^6} \begin{cases} (h - r)^3, & 0 \leq r \leq h, \\ 0, & \text{otherwise} \end{cases}$$

$$w_{\text{viscosity}}(\mathbf{r}, h) = \frac{15}{2\pi h^3} \begin{cases} -\frac{r^3}{2h^3} + \frac{r^2}{h^2} + \frac{h}{2r} - 1, & 0 \leq r \leq h, \\ 0, & \text{otherwise} \end{cases}$$

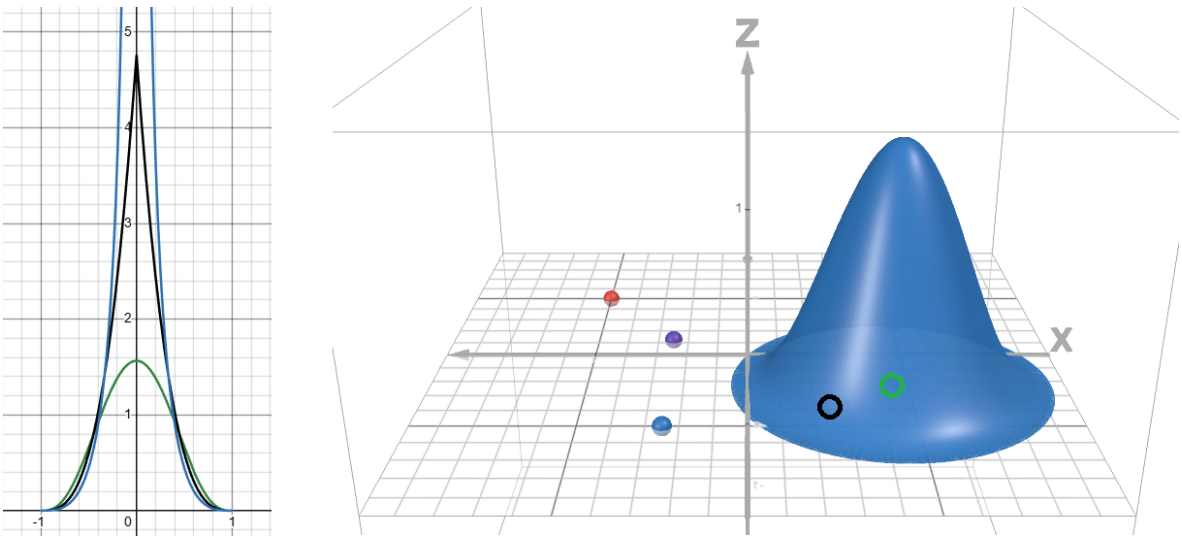

Figure 1: Smoothing kernels with radius 1 (left) - W_{poly6} green, W_{spiky} black and $W_{viscosity}$ blue. W_{poly6} kernel visualized in 3D with some points aligned on a plane (right).

2.5. Particle properties

Each particle has the following properties:

- Position
- Velocity

- Mass
- Density - density of particles in the surrounding area
- Pressure - difference between the pressure and target pressure

Position and velocity change as the fluid dynamics equations are evaluated. All of the particles have the same mass and it stays constant for the duration of the simulation. The density and pressure scalar fields are recalculated in each step of the simulation because their values depend on the nearby particles.

In the implementation, it is a good idea to save the non-constant values as fields (arrays) instead of saving them as member variables to some particle class. Each particle i has its attributes saved as the i-th element in each field. This representation helps when utilizing multi-threading or turning the computationally heavy functions into compute shaders for the GPU.

2.6. Density

To calculate the density ρ at the position of particle i:

$$\rho_i = \sum_j m_j \frac{\rho_j}{\rho_j} W(\mathbf{r}_i - \mathbf{r}_j, h) = \sum_j m_j W(\mathbf{r}_i - \mathbf{r}_j, h); j \neq i$$

The W_{poly6} kernel is used.

2.7. Pressure

To calculate the pressure force $-\nabla p$ at the position of particle i:

$$f_i^{pressure} = -\sum_j m_j \frac{p_j}{\rho_j} \nabla W(\mathbf{r}_i - \mathbf{r}_j, h); j \neq i$$

Ideally, the forces between some particles i and j ($i \neq j$) should be equal (but opposing). To achieve this, we can simply make the calculation symmetric by replacing p_j with $p_i + p_j$. This makes the calculation of the force equal for both particles i and j, with the only difference being the orientation of the force.

$$f_i^{pressure} = -\sum_j m_j \frac{p_i + p_j}{\rho_j} \nabla W(\mathbf{r}_i - \mathbf{r}_j, h); j \neq i$$

The first derivative of the W_{spiky} kernel is used.

2.8. Viscosity

To calculate the symmetric viscosity force $\mu \nabla^2 \mathbf{v}$ at the position of particle i:

$$f_i^{viscosity} = \sum_j m_j \frac{\mathbf{v}_i + \mathbf{v}_j}{\rho_j} \nabla^2 W(\mathbf{r}_i - \mathbf{r}_j, h); j \neq i$$

The second derivative of the $W_{viscosity}$ kernel is used

3. Implementation

The fluid simulation is implemented as a custom Actor class within the Unreal Engine environment. This allows for easy integration with the engine's rendering pipeline and scene hierarchy.

3.1. Spatial Hashing Grid

To address the quadratic complexity of neighbor searching, a spatial hashing grid is utilized. The 3D space is divided into virtual cells with a width equal to the smoothing radius h. Instead of checking every particle in the simulation, a particle only searches for neighbors within its own cell and the 26 adjacent cells.

The grid coordinates (x, y, z) for a particle position are calculated as:

$$x = \lfloor \text{pos.X}/h \rfloor, y = \lfloor \text{pos.Y}/h \rfloor, z = \lfloor \text{pos.Z}/h \rfloor$$

A unique hash key for each cell is generated using the following function:

$$\text{key} = (x \cdot 73856093) \oplus (y \cdot 19349663) \oplus (z \cdot 83492791)$$

This approach reduces the complexity from $O(N^2)$ to roughly $O(N \cdot M)$, where M is the average number of particles per neighborhood.

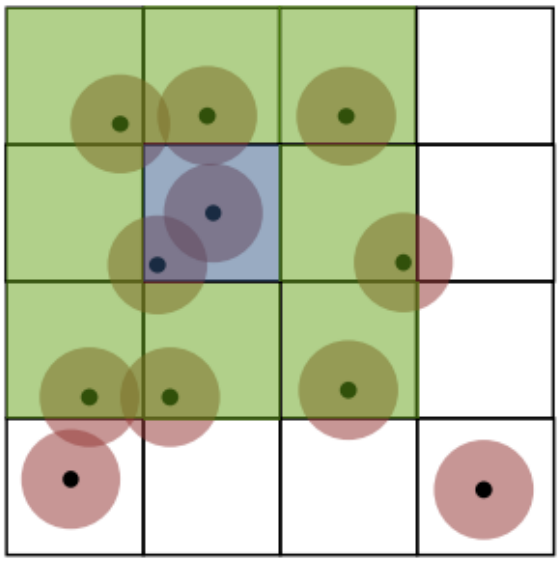

Figure 2: Example of spatial grid in 2D. We are looking for neighbors for particle in blue cell. We only need to search surrounding 8 cells, marked as green.

3.2. Predicted Positions and Near Pressure

A modification to the standard Müller SPH model is the use of predicted positions. Before calculating density and pressure, each particle's position is projected forward in time based on its current velocity:

$$\vec{r}_i^{\text{pred}} = \vec{r}_i + \vec{v}_i \cdot \Delta t$$

This predicted position is used for both the spatial grid update and the density field calculation. Additionally, a "near pressure" term is introduced to prevent particle overlapping. In the notation P_i^{near}, the P stands for **Pressure**. While standard pressure handles global fluid density and maintains the `TargetDensity`, near pressure acts as an aggressive short-range repulsion to keep particles from stacking on top of each other. It is calculated as:

$$P_i^{\text{near}} = \rho_i^{\text{near}} \cdot k_{\text{near}}$$

3.3. Smoothing Kernels

Different kernels are used for various parts of the simulation to ensure stability. Based on the implementation, the following smoothing functions $W(r, h)$ are used, where r is the distance and h is the `SmoothingRadius`:

- **Density Kernel (Spiky Pow2):** Used for calculating standard density.]

$$W_{\text{density}}(r, h) = \frac{15}{\pi h^5}(h - r)^2, \quad 0 \leq r \leq h$$

- **Near Density Kernel (Spiky Pow3):** Used for near-pressure calculation to provide sharper repulsion.

$$W_{\text{near}}(r, h) = \frac{15}{\pi h^6}(h - r)^3, \quad 0 \leq r \leq h$$

- **Viscosity Kernel (Poly6):** Utilized for viscosity force calculation to ensure smooth velocity distribution.

$$W_{\text{viscosity}}(r, h) = \frac{315}{64\pi h^9}(h^2 - r^2)^3, \quad 0 \leq r \leq h$$

The derivatives of these kernels are used to calculate the resulting forces. For instance, the density derivative used for pressure force is:

$$\nabla W_{\text{density}}(r, h) = \frac{30}{\pi h^5}(h - r), \quad 0 \leq r \leq h$$

3.4. Simulation Parameters

The following default values are used to achieve a stable waterfall effect:

- **Max Particles:** 3000
- **Particle Mass:** 520.0
- **Smoothing Radius (h):** 22.0
- **Target density:** 1.0
- **Gravity Force:** 500.0
- **Pressure Multiplier (k):** 50000.0
- **Near Pressure Multiplier (k_{near}):** 30000.0
- **Collision Damping:** 0.5
- **Viscosity Multiplier:** 1.0

3.5. Interaction with Objects

Collision detection is handled through two main systems:

- **Boundary Box:** Defined by the `BoundaryMesh` actor. Its mesh is hidden during runtime, and a red debug wireframe is drawn to represent the limits. If the actor is not assigned, deault values $2000 \times 2000 \times 2000$ are used. If a particle leaves the volume, it is pushed back, and its velocity is reflected and damped.
- **Obstacle Boxes:** An array of actors (`BoxObstacles`) acts as solid obstacles. If a particle enters the Axis-Aligned Bounding Box (AABB) of an obstacle, it is moved to the nearest face of that box, and its velocity is updated to reflect the bounce.

3.6. Particle Emitter

The emitter spawns particles at the `SpawnPoint` location. Particles are distributed in a grid pattern and managed using a circular array[1, 2]. Each particle has an `IsActive` flag; particles are initially spawned deep below the scene and only activated and moved to the spawn point when needed.

3.7. Rendering with Instanced Static Meshes

To render thousands of particles efficiently, an `InstancedStaticMeshComponent` (ISM) is used. All particle instances are initialized once during the start of the simulation. In every frame, the simulation updates the transform (position and scale) of each active instance and marks the render state as dirty to trigger a single draw call for all particles.

3.8. Fixed Time Step

The simulation utilizes a fixed time step of 1/60s. This is necessary to ensure that physical interactions remain consistent regardless of the current frame rate, preventing "explosions" of particles when the frame rate drops.

3.9. Simulation Pipeline

The simulation follows these steps in every update:

1. Spawn new particles at the frequency defined by `SpawnFrequency`.
2. Apply gravity and calculate **predicted positions** based on current velocities.
3. Update the **spatial grid** using the predicted positions.
4. Calculate **density** and **near-density** fields.
5. Compute **pressure** and **viscosity forces** using predicted positions.
6. Update **velocities** and integrate to find the final positions.
7. Resolve **collisions** with the boundary and obstacles.
8. Update the **ISM transforms** for rendering.

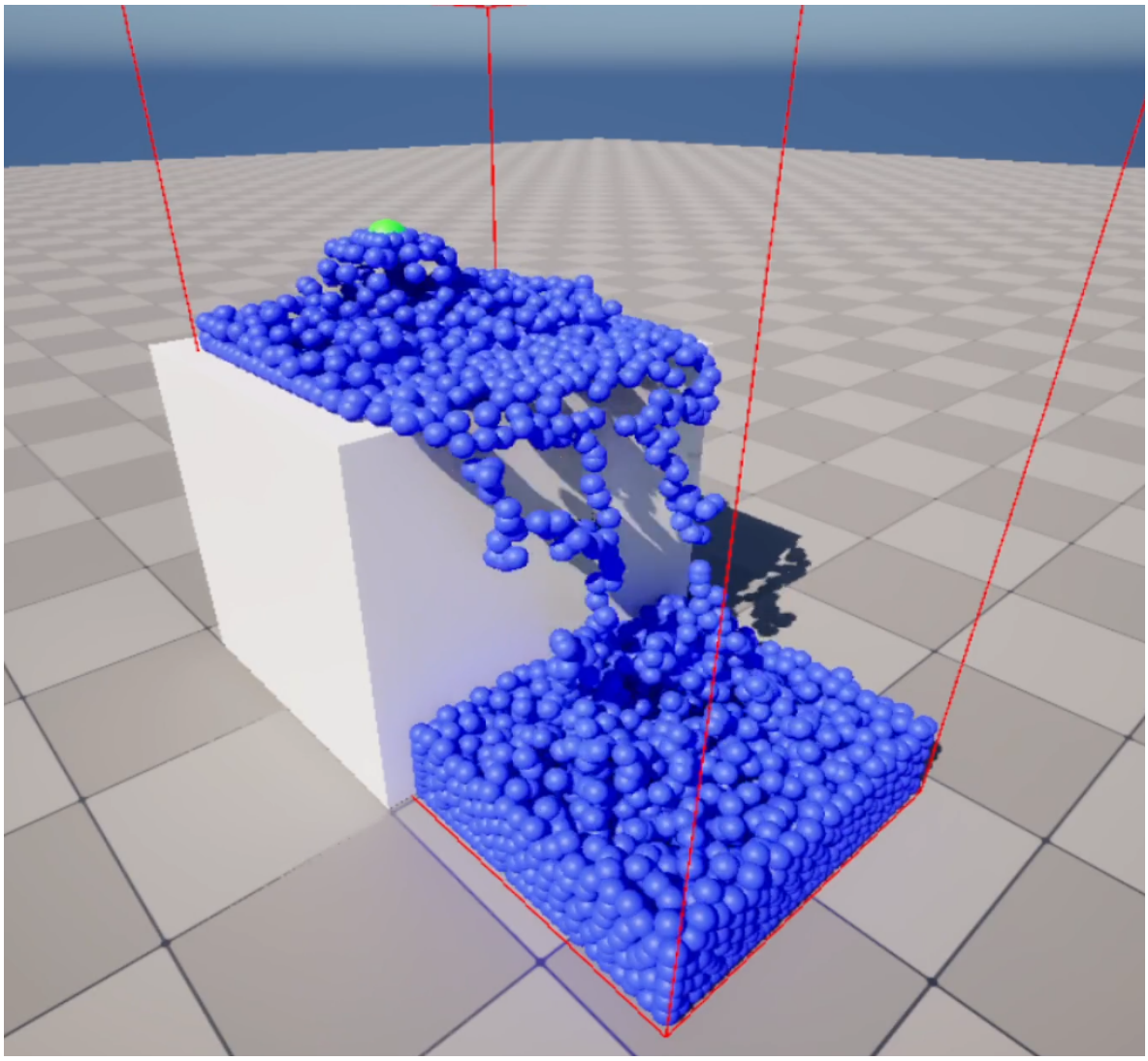

Figure 3: Image of our implemented waterfall simulation.

4. Conclusions

The implementation of the SPH-based fluid simulation in Unreal Engine successfully achieved the goal of simulating a dynamic waterfall in real-time. Several key technical decisions contributed to the stability and visual quality of the resulting simulation.

The combination of the **Spiky kernel** for density and pressure calculations, paired with the additional **near pressure** term, proved essential in maintaining particle separation. This approach effectively prevented the numerical instability caused by particle overlapping, which is a common issue in standard SPH implementations. Furthermore, the use of a **fixed time step** ensured that the physical forces remained predictable and stable, regardless of the rendering performance of the engine.

Fine-tuning the simulation parameters was a complex and iterative process. A systematic approach was adopted to achieve a balanced fluid state:

- The desired particle spacing was established to define the initial resolution of the fluid.
- The `TargetDensity` was set to 1.0 to serve as the baseline for pressure calculations.
- The simulation was initially tested without gravity to balance `ParticleMass` and `PressureMultiplier`, ensuring the particles reached a stable equilibrium.
- Gravity was subsequently introduced to the system.
- The `PressureMultiplier` was further refined, and the `NearPressureMultiplier` was added to handle high-compression scenarios.
- Finally, `CollisionDamping` and the `ViscosityMultiplier` were tuned to give the water its characteristic flow and energy loss upon impact.

The final `WaterfallScene` demonstrates these principles effectively. It features a central obstacle over which particles are spawned, flowing downwards to mimic a natural waterfall. Due to the interaction of pressure and viscosity forces, the particles naturally form small streams as they flow over the edges, providing a convincing representation of surface tension and fluid cohesion. The use of spatial hashing ensures that this complexity is handled efficiently, maintaining high performance even as the particle count reaches its maximum limit.

References

[1] Müller, M., Charypar, D., & Gross, M. Particle-based fluid simulation for interactive applications (2003) `https://matthias-research.github.io/pages/publications/sca03.pdf`

[2] Lague S. Coding Adventure: Simulating Fluids (2023) `https://www.youtube.com/watch?v=rSKMYc1CQHE`

[3] Image of spatial grid in 2D `https://www.marckletz.at/GPU_SPH.html`